

It also does not attempt to solve problems of encapsulation. The inherited structure of a class depends on the names of the internal parts of the classes from which it inherits. CLOS does not support subtractive inheritance. Within Common Lisp, there is a primitive module system that can be used to help create separate internal name-spaces.

Classes

The *defclass* macro is used to define a new class. The definition of a class consists of its name, a list of its direct super-classes, a set of slot specifiers and a set of class options. The direct super-classes of a class are those from which the new class inherits structure and behaviour. When a class is defined, the order in which its direct super-classes are mentioned in the *defclass* form defines a local precedence order on the class and those super-classes. The local precedence order is represented as a list consisting of the class, followed by its direct super-classes, in the order mentioned in the *defclass* form. The following two classes define a representation of a point in space. The *x-y-position* class is a sub-class of the *position* class:

```
> (defclass position () ())

> (defclass x-y-position (position)
  ((x :initform 0)
   (y :initform 0))
  (:accessor-prefix position-))
```

The *position* class is useful if we want to create other sorts of representations for spatial positions. The *x* and *y* coordinates are initialised to 0 in all instances, unless explicit values are supplied for them. To refer to the *x* coordinate of an instance of *x-y-position*, you would write:

```
> (position-x position)
```

To alter the *x* coordinate of that instance, you would write:

```
(setf (position-x position) new-x)
```

The macro *defclass* is part of the Object System programmatic interface and, as such, is on the first of the three levels of the Object System.

Generic functions

The class-specific operations of the Common Lisp Object System are provided by generic functions and methods. A generic function is one whose behaviour depends on the classes or identities of the arguments supplied to it. The methods associated with the generic function define the class-specific operations of the generic function.

Like an ordinary Lisp function, a generic function takes arguments, performs a series of operations and returns values. An ordinary function has a single body of code that is always executed when the function is called. A generic function is able to perform different series of operations and to combine the results of the operations in different ways, depending on the class or identity of one or more of its arguments.

Generic functions are defined by means of the *defgeneric-options* and *defmethod* macros. The *defgeneric-options* macro is designed to allow for the specification of properties that pertain to the generic function as a whole, and not just to individual methods. The *defmethod* form is used to define a method. If there is no generic function of the given name, however, it automatically creates a generic function with default values for the argument precedence order (left-to-right, as defined by the *lambda-list*), the generic function class (the class *standard-generic-function*), the method class (the class *standard-method*) and the method combination type (*standard-method* combination).

Methods

The class-specific operations provided by generic functions are themselves defined and implemented by methods. The class or identity of each argument to the generic function indicates which method or methods are eligible to be invoked.

A method object contains a method function, an ordered set of parameter specialisers that specify when the given method is applicable, and an ordered set of qualifiers that are used by the method combination facility to distinguish between methods.

The *defmethod* macro is used to create a method object. A *defmethod* form contains the code that is to be run when the arguments to the generic function cause the method that it defines, to be selected. If a *defmethod* form is evaluated, and a method object corresponding to the given generic function name, parameter specialisers and qualifiers already exists, then the new definition replaces the old.

Generic functions can be used to implement a layer of abstraction on top of a set of classes. For example, the *x-y-position* class can be viewed as containing information in polar coordinates.

Two methods have been defined—*position-rho* and *position-theta*, that calculate the ρ and θ coordinates given an instance of *x-y-position*:

```
> (defmethod position-rho ((pos x-y-position))
  (let ((x (position-x pos))
        (y (position-y pos)))
    (sqrt (+ (* x x) (* y y)))))

> (defmethod position-theta ((pos x-y-position))
  (atan (position-y pos) (position-x pos)))
```